

CMPS 357 – Team 3 Final Project Proposal

1. Project Title and Project Information

Project Title: RiskRadar Web – Personalized Environmental Risk Intelligence Platform

Group Members: Rebecca Gautreaux, Max Compeaux

Repository URL:

<https://github.com/School-of-Computing-and-Informatics/cmsp-357-sp26-final-project-cmps357-team-3.git>

GitHub Classroom Assignment: <https://classroom.github.com/a/oHRMfboi>

2. Project Overview

RiskRadar Web is a web-based extension of the RiskRadar mobile app, designed to help users better understand environmental hazards affecting their health, activities, and travel plans. Building on the existing app's ability to aggregate weather, air quality, and pollution alerts, the web platform introduces a **Personalized Environmental Risk Scoring Engine** and a **Smart Alert Prioritization System** that analyze environmental data and compute individualized risk scores based on each user's health sensitivities and preferences. Alerts are then dynamically ranked by risk score, geographic proximity, and hazard severity, transforming complex environmental data into clear, actionable insights that help users make safer decisions about outdoor activity and travel.

3. Project Goals

Core Capabilities: The completed system will include a web-based dashboard displaying RiskRadar backend data, a Personalized Environmental Risk Scoring Engine (dynamic scores based on real-time conditions and user sensitivities), and a Smart Alert Prioritization System ranking alerts by risk, proximity, and severity. Users can define personal sensitivities (e.g., asthma, pollen, heat) and view prioritized alerts with explanations. The app will integrate with existing RiskRadar APIs and pipelines and provide visual summaries to help users interpret their personal risk level.

Time-Permitting Features: Predictive risk forecasting, an interactive geographic risk map, and an AI-based environmental assistant.

4. Key Features

Personalized Environmental Risk Scoring Engine – Converts raw environmental data into a **user-specific risk score** (0–100) reflecting how dangerous current conditions are for a given individual. Incorporates **algorithmic risk modeling**, **multi-source data aggregation**, **personalized analytics**, and **explainable scoring logic**.

Smart Alert Prioritization System – Ranks alerts by relevance and urgency using three factors: **contribution to the user's risk score**, **geographic proximity to the hazard**, and **alert severity**. Requires **geospatial calculations**, **severity weighting**, **dynamic ranking**, and **personalized filtering**.

Web Dashboard – A centralized interface displaying the **user's risk score**, **ranked alerts**, **risk factor explanations**, and **environmental data trends**. Designed for clarity so users can quickly interpret conditions and understand their personal risk.

Data Storage and Processing – Integrates with the existing RiskRadar backend, extending it with models for **user sensitivity profiles**, **calculated risk scores**, and **prioritization metrics**. Environmental data continues to flow through existing automated scraper pipelines.

5. Technical Stack (Proposed)

Frontend: A new web-app built with PHP for server-side rendering, featuring **dynamic page rendering**, **backend API integration**, **responsive HTML/CSS styling**, and dashboard components displaying **risk scores**, **alert rankings**, and **environmental summaries**.

Backend: Built on the existing RiskRadar Python/FastAPI infrastructure, which handles **environmental data aggregation**, **risk scoring**, **alert prioritization**, and **REST API endpoints for the dashboard**.

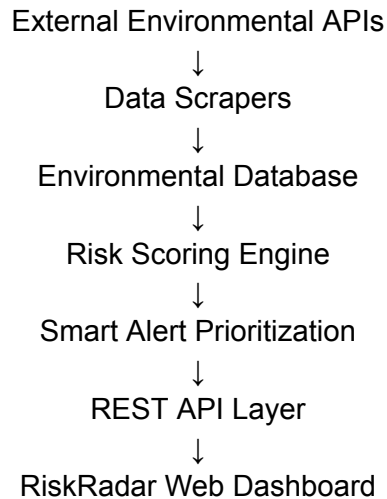
Database: Uses the existing MySQL setup, ensuring compatibility with current data pipelines and scrapers. Stores **user profiles**, **sensitivity preferences**, **risk scores**, **alerts**, and **hazard data**. MySQL's geographic data types also support proximity-based hazard calculations.

External APIs & Data Sources: **Environmental data** (air quality, weather alerts, pollution monitoring) is pulled from existing RiskRadar scrapers, which will be reused for the web platform.

6. Architecture Overview

System Interaction: Environmental data is automatically collected from external APIs, stored in a database, and processed by a Risk Scoring Engine that generates personalized user risk scores. A Smart Alert Prioritization System then ranks alerts based on those scores, while a FastAPI-powered REST backend connects everything to a web frontend that dynamically displays the processed data, risk scores, user profiles, and alerts on a dashboard.

System Architecture Diagram:



This layered architecture separates responsibilities between data collection, risk analysis, alert prioritization, and user presentation, improving maintainability and scalability.

7. Development Timeline

Week	Dates	Tasks	Key Deliverable
1	Mar 16 – Mar 22	Setup & Planning — create repository, review RiskRadar backend, design DB schema extensions, define API endpoints	RiskRadar API & pipeline integration
2	Mar 23 – Mar 29	Risk Engine — implement environmental data aggregation, develop initial risk scoring algorithm, begin backend integration	Personalized Risk Scoring Engine
3	Mar 30 – Apr 5	Alert Prioritization — implement alert ranking algorithm, integrate severity/proximity calculations, develop backend API endpoints	Smart Alert Prioritization System
4	Apr 6 – Apr 12	Dashboard MVP — build initial React dashboard, display risk scores and prioritized alerts	Web dashboard displaying RiskRadar hazard data
	Checkpoint (Apr 8)	Demonstrable: functioning Risk Scoring Engine + working dashboard	
5	Apr 13 – Apr 19	Personalization — implement user sensitivity profiles, integrate into risk calculations, add	User sensitivity profile system; alert

		explanatory alert summaries	explanations
6	Apr 20 – Apr 26	UI Polish & Testing — refine dashboard layout, implement data visualizations, system testing and debugging	Visual risk summaries and interpretive UI
7	Apr 27– May 1	Finalization — documentation, bug fixes, deployment prep, and final presentation	All core deliverables complete (<i>Target: Apr 29</i>)

8. Stretch Goals

Interactive Risk Map (Plotly): A geographic map visualizing **air quality zones**, **wildfire smoke**, **pollution alerts**, and **weather warnings**. The backend generates visuals using Plotly (leveraging skills from Programming Assignment 2), with MySQL's geographic data types supporting proximity-based hazard calculations.

Predictive Environmental Risk Forecasting (AI/Data Extension): Forecasts risk levels 24 to 48 hours out using **historical data** (air quality, temperature, wind speed, pollution trends), **incorporating time-series analysis** and **predictive modeling**.

AI Environmental Assistant: An integrated assistant that helps users interpret data and alerts by answering **natural language questions** (e.g., "Is it safe to exercise outdoors?", "Why is my risk score high?"). Responses are generated using **live environmental data**, **alerts**, and **risk scores** as context.

9. Why This Project Matters

RiskRadar Web extends an existing mobile app with an analytical web platform, demonstrating full-stack development, API design, data pipelines, and algorithmic decision systems. It reflects modern practices — modular architecture, user-centered design, and AI integration — to transform raw environmental data into personalized insights, detailed alert rankings, and interactive visualizations within a larger dashboard environment.

As environmental hazards grow more frequent and complex, RiskRadar Web illustrates how software can empower users to make safer, data-driven decisions — making it both technically meaningful and practically relevant.

Written with assistance from ChatGPT, GitHub Copilot, and ClaudeAI.

<https://chatgpt.com/share/69b0315d-8724-8002-a602-de27293427b8>

<https://claude.ai/share/15d67062-8b56-454c-88e5-b6979951cd0d>